CN & SCALER · SESSION 5

HTTP, 1996 → now

Same words. Three completely different ways of writing them down.

```
$ printf 'GET / HTTP/1.0\r\n\r\n' | nc localhost 8010

# ... and in 2 hours, the same request over QUIC
```

RFC 1945 · RFC 2068 · RFC 2616 · RFC 7230 · RFC 9110 · RFC 9113 · RFC 9000 · RFC 9114

Week	Broad Topic
Week 1	Network programming 101
Week 2	Computer Networking Eagle Eye View
Week 3	Server Side Considerations for scale
Week 4	nginx deep dive
Week 5	Evolution of HTTP
Week 6	(Ab)using CDNs
Week 7	Building for failures
Week 8	Streaming video at large scale

EARLY HTTP/1.1 · ASSIGNMENT · DUE BEFORE SESSION 7

Build a calculator that stays on the line

The task

any language, no framework, just a socket

```
GET /add?a=2&b=3    ->  200   5
GET /sub?a=10&b=4   ->  200   6
GET /mul?a=6&b=7    ->  200   42
GET /div?a=9&b=3    ->  200   3

GET /div?a=1&b=0    ->  400
GET /add?a=x&b=3    ->  400
GET /pow?a=2&b=8    ->  404
POST /add           ->  405
GET /add  (no Host)   ->  400
```

That is the entire feature set. You could write it in an afternoon, and the arithmetic is not the point.

Don't read 02-http11/server11.py

How I will mark it

one socket. every request.

```
s = socket.create_connection(("localhost", 8080))

  GET /add?a=2&b=3      -> 200   5
  GET /sub?a=10&b=4     -> 200   6
  GET /mul?a=6&b=7      -> 200   42
  GET /div?a=1&b=0      -> 400
  GET /pow?a=2&b=8      -> 404
  POST /add             -> 405

  socket still open: True
  1 TCP handshake, 6 responses
```

If the socket dies before I am done,

The part that is actually hard. Keeping the connection open forces a question HTTP/1.0 never asked you: **where does this request end and the next one begin?** While you were hanging up, the answer was "at EOF", for free. Now you must consume exactly Content-Length bytes and not one more — byte n+1 belongs to somebody else.

Stretch, all optional: honour Connection: close · an idle timeout you can defend · chunked encoding · take all six at once and answer in order, which is pipelining.

▶ Now you write the spec

Track 1 — the server

```
$ ./bserve ./www 9000
```

```
accept a TCP connection
read one binary request frame
map the path to a file under a root
reply: status, headers, the bytes
404 if it is not there
400 if the frame is malformed
and keep the connection open
```

Track 2 — the client

```
$ ./bcurl -v localhost:9000/index.html
```

```
build the binary request frame
read the response, body to stdout
-v hexdumps every frame
exit non-zero on 4xx / 5xx

and never open a second connection
```

The bit in the middle is the actual project

A fixed-size frame header — you pick the fields and the widths, and you defend them. HTTP/2 chose 24 / 8 / 8 / 31. Why?

Headers: number the ten names you actually send, length-prefix the rest — HPACK's first two mechanisms, in an evening.

And one line you may not skip: a receiver meeting a frame type it does not know MUST skip it cleanly. That is how you leave room for a version 2.

What you hand in

1. The spec. Two pages. Enough for a stranger.
2. Your program.
3. An annotated hexdump of one complete request and response.

If you cannot annotate your own bytes, the spec is not finished.

In pairs: one server, one client, and the only thing that crosses between you is the spec. A client that only works against your own server is an implementation, not a protocol.

▶ The whole protocol fits on one slide

Methods

GET **HEAD** **POST**

PUT DELETE LINK UNLINK

Appendix D: "additional methods"

that exist but are not in the spec.

Headers

Allow	Authorization
Content-Encoding	Content-Length
Content-Type	Date
Expires	From
If-Modified-Since	Last-Modified
Location	Pragma
Referer	Server
User-Agent	WWW-Authenticate

Status codes

200 OK	201 Created
202 Accepted	204 No Content
300 Multiple Choices	
301 Moved Permanently	
302 Moved Temporarily	
304 Not Modified	
400 Bad Request	401 Unauthorized
403 Forbidden	404 Not Found
500 Internal Server Error	
501 Not Implemented	502 Bad Gateway
503 Service Unavailable	

Today: 62 assigned. (IANA, Sep 2025)

datatracker.ietf.org/doc/html/rfc1945 · IANA HTTP Status Code Registry, updated 2025-09-15

▶ Speak it by hand, once

```
$ python3 tools/talk.py 8010 --req "GET / HTTP/1.0" ""
--- sent 18 bytes

HTTP/1.0 200 OK
Date: Fri, 04 Sep 2026 07:11:27 GMT
Server: server10/1.0 (RFC 1945)
Content-Type: text/html
Content-Length: 263
Last-Modified: Fri, 04 Sep 2026 07:11:14 GMT
Expires: Fri, 04 Sep 2026 07:12:27 GMT
                                  <- the blank line is the delimiter
<!doctype html>
<title>site-a</title>
...

--- server closed the connection (FIN) at 2.9 ms
```

Not curl. curl adds four headers you did not write and hides two the server sent.

01-http10/server10.py · tools/talk.py

One response. Then the socket dies.

RFC 1945 §1.3

"current practice requires that the connection be established by the client prior to each request and closed by the server after sending the response."

A 24-object page, 200 ms RTT:

```
24 TCP handshakes    = 4.8 s
24 request/response  = 4.8 s
                      ------
                       9.6 s
```

…and 24 congestion windows that each start from nothing.

Try to reuse the socket

```
it is not there any more
s.sendall(b"GET /style.css HTTP/1.0\r\n\r\n")
r1 = recv_until_eof(s)
# request 1: 274 bytes, server sent FIN

# now try a second request on the SAME socket
s.sendall(b"GET /app.js HTTP/1.0\r\n\r\n")
r2 = s.recv(4096)

# request 2: b''
```

Nothing. Not an error — an empty read. The connection was closed the moment the first response finished.

▶ No Host header. One IP, one website.

```
$ talk.py 8010 --req "GET / HTTP/1.0" "Host: site-b.example" ""

HTTP/1.0 200 OK
<h1>site-a</h1>            <- not site-b. it never had a choice.

# server log:
  (ignoring Host: site-b.example - HTTP/1.0 has no vhosts)
```

1 : 1

sites per IP address, until January 1997

~1.1 B

websites today. Without Host, a third of all of IPv4.

SNI

the same problem one layer down, once TLS hid the Host

The fix — a single header field — arrives nine months later and is arguably the highest-leverage line in the whole standard.

▸ Caching, with a clock as the fingerprint

```
$ talk.py 8010 --req "HEAD /style.css HTTP/1.0" ""
Last-Modified: Fri, 04 Sep 2026 07:11:14 GMT

$ talk.py 8010 --req "GET /style.css HTTP/1.0" \
      "If-Modified-Since: Fri, 04 Sep 2026 07:11:14 GMT" ""

HTTP/1.0 304 Not Modified
                          <- no body. 202,000 bytes -> 0.
```

Three things wrong with a timestamp

1 s

No sub-second field. Two writes in one second are one write to every cache alive.

drift

Your clock, my clock, the proxy's clock. The spec has to legislate for disagreement.

≠

mtime is not identity. A rebuild changes every mtime and none of the bytes.

RFC 1945 §10.7 Expires · §10.9 If-Modified-Since · §10.10 Last-Modified

▶ What HTTP/1.0 could not do

The limitation	Concretely	Fixed in
Connection is not persistent	One request per TCP handshake, forever	RFC 2068 §8.1
Sixteen headers, three methods	No OPTIONS: you cannot ask what a server supports	RFC 2068 §9.2
Caching exists but is thin	Expires + If-Modified-Since. A clock, not a fingerprint	RFC 2068 §14.19
No Host	One IP address per website	RFC 2068 §14.23

One correction while we are here. HTTP/1.0 is usually listed as having "head-of-line blocking". It does not, quite. Head-of-line blocking needs a queue, and HTTP/1.0 has no pipelining to queue anything on. What it has is a serialisation cost: one round trip per object, no exceptions.

Real head-of-line blocking arrives in HTTP/1.1, as a side effect of the fix for this. Watch for that pattern all day — every solution today is the cause of the next slide.

▶ Persistence became the default. Now you opt out.

HTTP/1.0

`ask, and maybe receive`

`Connection: keep-alive`

A non-standard extension. Some servers honoured it, some ignored it, some crashed. Documented in RFC 2068 §19.7.1 as a compatibility note, not a feature.

The default was: hang up.

HTTP/1.1

`RFC 2616 §8.1.2`

`Connection: close`

"persistent connections are the default behavior of any HTTP connection."

Every implementation that did nothing at all got the fast path.

The default was: stay.

```
$ 02-http11/demos.sh   # demo 1
  /           HTTP/1.1 200 OK   socket still open: True
  /style.css  HTTP/1.1 200 OK   socket still open: True
  /app.js     HTTP/1.1 200 OK   socket still open: True
  three responses, one TCP handshake.
```

One header. One IP address, a thousand sites.

```
$ talk.py 8011 --req "GET / HTTP/1.1" ""        # no Host
HTTP/1.1 400 Bad Request
HTTP/1.1 requires a Host header (RFC 2616 14.23).

$ ... "Host: site-a.local"   ->  <h1>site-a</h1>
$ ... "Host: site-b.local"   ->  <h1>site-b</h1>
                                 same IP. same port. same socket.
```

The same move, four layers, thirty years

Layer	The problem	The fix
HTTP/1.0	The four-tuple was the address; one IP = one site	Host: — put the name in the message
TLS	Host is encrypted; the cert must be chosen first	SNI in the ClientHello, in the clear
HTTP/2	Headers are compressed and unordered	:authority as a pseudo-header
Everywhere else	Many logical destinations, one connection	Topic. Routing key. Ingress host rule.

A fingerprint, not a clock

```
$ talk.py 8011 --req "HEAD /big.txt HTTP/1.1" "Host: site-a.local" ""
ETag: "55274ac22c651b90"          <- sha256 of the bytes, first 16 hex

$ talk.py 8011 --req "GET /big.txt HTTP/1.1" \
      "Host: site-a.local" "If-None-Match: \"55274ac22c651b90\"" ""

HTTP/1.1 304 Not Modified
ETag: "55274ac22c651b90"
--- received 256 bytes
```

202,000 → 0

bytes of body. The 200 would have carried the whole file; the 304 carries nothing.

W/"…"

a weak validator: equivalent, not identical. Range requests need a strong one.

If-Match

the same idea pointed at writes: optimistic concurrency, and the fix for lost updates.

RFC 9110 §8.8.1 strong vs weak · §13.1.1 If-Match · §13.1.2 If-None-Match

▶ Accept-*, and the header that keeps caches honest

```
$ 02-http11/demos.sh   # demo 7
  Accept-Encoding: identity  -> Content-Length: 202000  Vary: Accept-Encoding
  Accept-Encoding: gzip      -> Content-Length: 5763    Content-Encoding: gzip
  Accept-Encoding: deflate   -> Content-Length: 5751    Content-Encoding: deflate
```

2.8%
of its original size.

Which quietly undercuts "HTTP/1.1 is slow because it is text". Text compresses. The body was never the problem — the **headers** were, because Content-Encoding compresses the body only.

Vary is a cache-key instruction

It says: this URL has more than one answer, and here is what the answer depends on.

`Vary: Accept-Encoding` correct
`Vary: User-Agent` millions of keys, hit rate → 0
`Vary: *` caching off entirely

Range: the header that runs your data lake

```
Range: bytes=100-199     ->  HTTP/1.1 206 Partial Content
                             Content-Range: bytes 100-199/202000
                             Content-Length: 100

Range: bytes=-50         ->  Content-Range: bytes 201950-201999/202000
                             the last 50 bytes, size not known in advance

Range: bytes=999999-     ->  HTTP/1.1 416 Range Not Satisfiable
                             Content-Range: bytes */202000
```

What one header bought

Video scrubbing	Resumable downloads	Parquet / ZIP footers
The player asks for the bytes around the timestamp you clicked — not the film.	Died at 60%? Range: bytes=600000000- and carry on where you stopped.	The directory is at the END. Read the last few KB, then fetch only the columns you need.

416 was not in RFC 2068 — RFC 2616 added it to separate "your syntax is wrong" from "that file is smaller than you think".

▶ Send a body without knowing how long it is

```
$ talk.py 8011 --req "GET /style.css?chunked=1 HTTP/1.1" ...

HTTP/1.1 200 OK
Content-Type: text/css
Transfer-Encoding: chunked
                                <- and NO Content-Length

37
body{font-family:monospace;background:#111;color:#eee}
0
```

Session 2, closing idea 2: framing is length or delimiter — there is no third option.

Chunked encoding is both, nested: each chunk is length-prefixed, and the sequence is delimited by an empty one. That is FastCGI's record structure from Session 3, exactly. Same shape, different decade — and it is what streaming JSON, server-sent events and gRPC trailers are all standing on.

And the rest of it, in one breath

Arrival	What it is for	Where you meet it now
OPTIONS	Ask what a server supports	Every CORS preflight your browser sends
PUT · DELETE	Write and remove, not just read	The R and D of every REST API since
TRACE	Echo my request back, proxies and all	Disabled everywhere — Cross-Site Tracing, 2003
Via	Each proxy appends itself	Working out which box in the chain is lying
Expect: 100-continue	Ask before uploading four gigabytes	curl's mysterious 1-second pause on big PUTs
Cache-Control	A vocabulary instead of one timestamp	max-age, s-maxage, no-cache ≠ no-store
~40 status codes	303, 405, 409, 410, 412, 413, 415, 504…	504 is here already — not a 1999 addition

no-cache means revalidate before use. no-store means do not write it down. Everybody gets this wrong exactly once.

▶ "SHOULD NOT maintain more than 2 connections"

```
$ python3 03-limits/fetch_page.py --port 9111 --rtt 150

  6 assets, RTT 150 ms
  mode                    bytes  round trips  TCP conns   wall ms
  close       (HTTP/1.0)   4867           12          6    1369.2
  keepalive   (1 conn)     5095            7          1     985.4
  parallel2   (RFC says)   5095            4          2     607.0
  parallel6   (browsers)   5095            2          6     608.2
  pipelined   (1 batch)    5095            2          1     303.7
```

6

connections per origin, what every browser actually shipped by about 2008

18

via domain sharding — img1, img2, img3. A hack on top of a violated rule.

dropped

RFC 7230 removed the sentence in 2014. The spec conceded.

The best row in that table is the one nobody shipped. Next slide.

▶ 1999: mostly tightening, not adding

Genuinely new

§19.6.3, "Changes from RFC 2068"

307 Temporary Redirect
416 Range Not Satisfiable
417 Expectation Failed + Expect:

identity content-coding
TE: and a transfer-coding registry
Host is now a MUST (and a MUST-400)
ETag required on resource creation
charset wildcards in Accept-*
byte ranges without a known length

NOT new: 504 Gateway Timeout.
It is in RFC 2068 §10.5.5 already —
the whole 5xx family arrived in 1997.

Why 307 exists

a standard losing to deployed reality

RFC 2068 said: 302 keeps the method.

Every browser turned the POST into
a GET anyway.

RFC 2616 could have said "no, really".
Instead it renamed 302 to "Found",
wrote down what implementations do,
and added 307 for people who meant
what 302 originally said.

It is easier to add a number than to fix a billion deployed clients.

▸ Watch Postel's law lose an argument

RFC 2616 §4.4 · June 1999

`be liberal in what you accept`

"If a message is received with both a Transfer-Encoding header field and a Content-Length header field, the latter MUST be ignored."

Recover. Do not reject.

In 1999 this was simply what good engineers did.

RFC 9112 §6.3 · June 2022

`the same situation, 23 years later`

"A server MAY reject a request that contains both … or process such a request in accordance with the Transfer-Encoding alone. **Regardless, the server MUST close the connection after responding.**"

Rejection is now allowed. Recovery is now optional. And either way, destroy the connection.

Be liberal in what you accept is a fine rule for two programs. It is a catastrophic rule for a chain of five.

One byte stream. Two readings. One attack.

one stream, sent once

```
POST /echo HTTP/1.1
Host: site-a.local
Content-Length: 6
Transfer-Encoding: chunked

0

GET /admin HTTP/1.1
Host: site-a.local
```

Reading it with Content-Length: 6

ONE request. Its 6-byte body is 0\r\n\r\nG

Reading it with Transfer-Encoding

TWO requests. The second is GET /admin

Put a CDN that does the first in front of an origin that does the second, and you have posted a request the CDN never saw — and left a spare response in the connection for whoever is routed onto it next.

```
$ python3 03-limits/framing.py --port 8011
  this server answered with 2 response(s):
    | HTTP/1.1 200 OK
    | HTTP/1.1 404 Not Found        <- the request nobody sent
```

HTTP/2 makes this unrepresentable: the length is in the frame header, and there is no second opinion to disagree with.

One document became six

RFC	Title	Who has to read it carefully
7230	Message Syntax and Routing	Proxies, load balancers, CDNs — and the https scheme lives here now
7231	Semantics and Content	Application frameworks: methods, status codes, representations
7232	Conditional Requests	Anything with an ETag: browsers, caches, object stores
7233	Range Requests	Video, downloads, Parquet readers
7234	Caching	Every CDN on earth
7235	Authentication	The WWW-Authenticate / Authorization framework

I jokingly call it “Microservices” for RFC. Same year, same argument: one large artefact with many stakeholders becomes several with clear ownership, versioned independently, so a change to caching does not require re-ratifying the definition of GET. Same trade-off too — 2616 was one place to look, and everyone still says "2616" when they mean HTTP/1.1.

▶ Why your JWT is base64

```
RFC 7230 §3.2.6, June 2014
  tchar = "!" / "#" / "$" / "%" / "&" / "'" / "*" / "+" / "-" / "." /
          "^" / "_" / "`" / "|" / "~" / DIGIT / ALPHA
  token = 1*tchar
  excluded:  " ( ) , / : ; < = > ? @ [ \ ] { }
```

Correction: this is not a 2014 rule

RFC 7230 only wrote it as a list. The rule is RFC 2616 §2.2 (1999), where token = any CHAR except CTLs or `separators` — and the separators list already contains { and }. Same rule, better prose, fifteen years earlier.

And the JWT reason is broader

A JWT must survive an Authorization header, a URL query string, a cookie, a form field and a JSON body. base64url also swaps + and / for - and _ so it is URL-safe — and the signature is raw bytes, which need encoding whatever the grammar says.

Every layer you pass through has an alphabet. The intersection of all of them is about sixty-four characters wide — which is exactly why base64 exists, and why Session 2 met the same problem in MIME (mail).

The split that matters, and a teapot

RFC	What it is	Version-specific?
9110	HTTP Semantics — methods, status, headers, ranges, auth	No. Shared by 1.1, 2 and 3.
9111	HTTP Caching	No. Shared by all three.
9112	HTTP/1.1	Wire format only
9113	HTTP/2	Wire format only
9114	HTTP/3	Wire format only

The table of contents is the argument

Everything HTTP ***means*** is in 9110 and 9111, shared by all three versions. 9112, 9113 and 9114 only describe how to write it down on a socket. Three encodings, one language.

418 (Unused)

An April Fool's joke (RFC 2324, 1998) that "has been widely implemented as an easter egg, and therefore is effectively consumed by this use." Reserved forever — not because it is useful, but because too many people shipped the joke.

Five fast responses, waiting 2 seconds for a sixth

Pipelined, one connection

all six requests sent at t=0

/big.txt?delay=2.0	2003.4 ms
/style.css	2003.6 ms
/app.js	2003.8 ms
/logo.png	2004.1 ms
/icon.png	2004.2 ms
/hero.png	2004.4 ms

Every one of those five was ready in microseconds. They waited two seconds for a file none of them needed.

One connection each

same six requests

/big.txt?delay=2.0	2006.0 ms
/style.css	3.2 ms
/app.js	5.7 ms
/logo.png	6.1 ms
/icon.png	6.9 ms
/hero.png	7.2 ms

Same server. Same files. Same work. The only difference is whether they had to queue behind each other.

▸ There is no request id on the wire

The order IS the identifier.

Nothing in an HTTP/1.1 response says which request it answers. The client knows only because it counted. Which makes order mandatory — and makes the slowest response set the pace for everything behind it.

Multiplexing is only possible if messages carry identity

Protocol	The identifier	Where you met it
TCP	source port / destination port	Session 2
SS7 / TCAP	transaction id	Session 2 — SMS and call setup
FastCGI	requestId, in every record header	Session 3 — php-fpm behind nginx
HTTP/1.1	— nothing —	and so: strict response ordering
HTTP/2 · HTTP/3	stream identifier, 31 bits, every frame	

▶ 97.5% of what you upload is a repeat

```
$ python3 03-limits/headers.py --requests 80

  one request, headers only              805 bytes
  x 80 requests for one page           64320 bytes
  new information (the paths)           1600 bytes     2.5%
  byte-for-byte repeats                62720 bytes    97.5%

  On a 1 Mbit uplink that is 0.51 s of upload before
  a single byte of your application data has moved.
```

805 B

of headers on one realistic request:
cookies, User-Agent, Accept-*, sec-fetch-*

upload

the direction you have least of, spent at
the start of the page, while the user watches

why not gzip?

the obvious answer, and the wrong one.

When keep-alive made it slower

Two algorithms, both correct

Nagle · RFC 896 · 1984

Do not send a small segment while an earlier one is unacknowledged.

Delayed ACK · RFC 1122

Do not acknowledge immediately; wait up to 40 ms to piggyback it.

```
write(head)   -> goes out, nothing unacked
write(body)   -> Nagle holds it
client        -> nothing to send, ACK delayed
  ... 40 ms ...
ACK, then body.  Every request but the 1st.
```

Measured, on loopback

03-limits/nagle.py

```
write(head); write(body)
  /style.css       0.9 ms
  /app.js         41.5 ms   <- delayed ACK
  /logo.png       44.0 ms   <- delayed ACK
  /icon.png       43.9 ms   <- delayed ACK
  /hero.png       44.0 ms   <- delayed ACK
  total          174.3 ms

one write, + TCP_NODELAY
  total            3.2 ms
```

It only appears on a PERSISTENT connection — close() pushes it out. The fix for one problem exposed it.

Your protocol sits on a protocol with its own opinions. nginx and Apache both coalesce and both set TCP_NODELAY — now you know why.

The bill for HTTP/1.1

The wall	Measured today	Fixed by
Head-of-line blocking	2003 ms vs 3 ms for the same file, pipelined	Stream ids — HTTP/2
Header bloat	97.5% of uploaded header bytes are repeats	HPACK — HTTP/2
Too many connections	6 per origin, 18 with domain sharding	One connection — HTTP/2
Ambiguous framing	One byte stream, two readings, one attack	Frame headers — HTTP/2
TCP itself	3 round trips before the first byte	QUIC — HTTP/3

"Text based — you die early, or you live long enough…" Text framing is why HTTP won: anyone could implement it, anyone could debug it with telnet. That beat every better-engineered rival of the 1990s.

But "text is slow" is not the finding — we measured gzip at 2.8%. The problem was text **without lengths and without identifiers**. HTTP/2 is not "HTTP but binary"; it is "HTTP but with lengths and ids", and binary is the consequence.

Ship it to a billion users, then standardise it

When	What	The number that made the case
Nov 2009	SPDY announced — Belshe & Peon, Chromium blog	"up to 55% faster", top 25 sites, in a lab
2010–2014	Chrome, Firefox, nginx, Twitter, Facebook ship it	Google runs both ends and measures production
Feb 2015	Google announces SPDY's retirement	>25% of Chrome resources on HTTP/2 vs <5% SPDY
May 2015	RFC 7540 (HTTP/2) + RFC 7541 (HPACK)	SPDY with the edges filed off
May 2016	SPDY removed in Chrome 51	six and a half years, experiment to obsolete
Jun 2022	RFC 9113 obsoletes 7540	deprecates priorities and the h2c Upgrade dance

It is called HTTP/2, not HTTP/2.0, on purpose. The working group dropped the minor version because "1.1" had produced a decade of confusion about what a 1.x server must support. There is no HTTP/2.1 and there will not be one — the next version is a new number and a new wire format.

▶ Twenty-four bytes that are also a joke

RFC 9113 §3.4 — the client connection preface

```
PRI * HTTP/2.0\r\n\r\nSM\r\n\r\n

50 52 49 20 2a 20 48 54 54 50 2f 32 2e 30 0d 0a
0d 0a 53 4d 0d 0a 0d 0a          exactly 24 octets, always
```

An HTTP/1.1 server that receives this sees method PRI, version HTTP/2.0, and fails cleanly with 501 — instead of trying to parse the binary frames behind it as text. (PRI…SM is what is left of "PRISM". It was 2013. The RFC does not say so.)

Over TLS — how it actually happens

RFC 9113 §3.2 · ALPN is RFC 7301

```
$ h2_client.py --port 8443 --tls
  TLS TLSv1.3  ALPN negotiated: 'h2'
```

The client lists h2 and http/1.1 inside the ClientHello; the server picks one in the ServerHello. Zero extra round trips — it rides in a handshake you already paid for.

Cleartext — and the dance that died

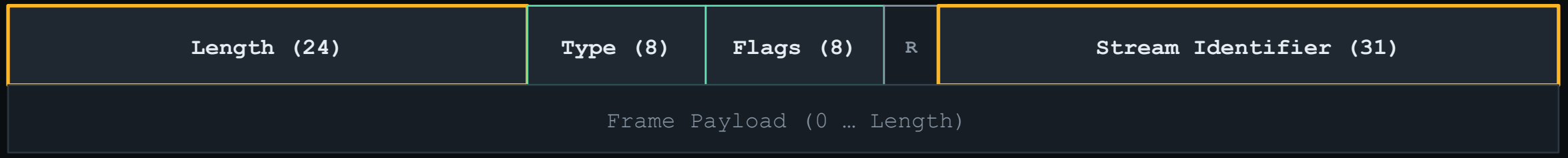
RFC 9113 §3.3 and §3.1

```
$ h2_client.py --port 8020
  h2c by prior knowledge
```

RFC 7540 had an Upgrade: h2c dance. RFC 9113 §3.1: it "was never widely deployed and is deprecated by this document." No browser ever shipped cleartext HTTP/2 at all.

Nine bytes, and 31 of them are the answer

31 bits that HTTP/1.1 never had



every frame header, raw, from a client we wrote by hand

```
$ python3 04-http2/h2_client.py --port 8020 --path /
  -> preface  b'PRI * HTTP/2.0\r\n\r\nSM\r\n\r\n'
  -> HEADERS stream=1 GET /   67B of text -> 15B hpack
  <- 00 00 2a 04 00 00 00 00 00  SETTINGS  len=42  stream=0
  <- 00 00 4d 01 04 00 00 00 01  HEADERS   len=77  stream=1  END_HEADERS
  <- 00 01 07 00 00 00 00 00 01  DATA      len=263 stream=1
  <- 00 00 00 00 01 00 00 00 01  DATA      len=0   stream=1  END_STREAM
```

Client streams are odd, server streams are even, so nobody has to coordinate (§5.1.1). Stream 0 is the connection itself — SETTINGS, PING, GOAWAY.

Request smuggling is not fixed here. It is unrepresentable: the length is in a fixed place, at a fixed size, with no second opinion.

Ten frame types, and what each replaced

Type	Name	What it does	What it replaced in HTTP/1.1
0x00	DATA	The body, in chunks ≤ MAX_FRAME_SIZE	Content-Length / chunked encoding
0x01	HEADERS	Opens a stream; carries an HPACK block	The request line and header block
0x02	PRIORITY	Deprecated by RFC 9113	→ RFC 9218 PRIORITY_UPDATE
0x03	RST_STREAM	Cancel ONE stream	Closing the TCP connection, killing everything
0x04	SETTINGS	Announce your limits, both directions	Nothing — there was no way to say
0x05	PUSH_PROMISE	Server push	Nothing — and it did not last (2 slides on)
0x06	PING	Round-trip time; keep intermediaries awake	Nothing
0x07	GOAWAY	"Last stream id I will process" — drain	Connection: close, mid-flight
0x08	WINDOW_UPDATE	Flow control, per stream AND per connection	TCP's window, and only TCP's
0x09	CONTINUATION	A header block bigger than one frame	Nothing — and a 2024 DoS class

§6.5.2 defaults: HEADER_TABLE_SIZE 4096 · INITIAL_WINDOW_SIZE 65535 · MAX_FRAME_SIZE 16384 · MAX_CONCURRENT_STREAMS has none ("no smaller than 100")

▶ 174 bytes → 79 → 6

Three mechanisms, none of them clever

1 Static table — 61 entries

Fixed, identical everywhere (App. A). Index 2 is ":method: GET" — name AND value. The whole of GET costs 0x82. Not compressed. Indexed.

2 Dynamic table — from index 62

Anything sent before on THIS connection gets an index. Your 300-byte cookie costs 300 bytes once, then 1–2 bytes forever.

3 Huffman — Appendix B

A canonical code built from real header text: "e" is 5 bits, "X" is 8. Per string, and only when it actually shrinks it.

Ours, against the RFC's own vectors

04-http2/hpack_mini.py

```
[ok] C.1.1  10 in a 5-bit prefix
[ok] C.1.2  1337 in a 5-bit prefix
[ok] C.4.1  huffman 'www.example.com'
[ok] C.6.1  decode response 1
[ok] round trip request 1
[ok] round trip request 2
[ok] pypi 'hpack' agrees with our encoder

the same header set, three ways:
  HTTP/1.1 text          174 bytes
  HPACK, first time       79 bytes  55%
  HPACK, second time       6 bytes  97%
```

Six bytes. Six one-byte indices for six header fields.

The dynamic table is per connection — which is why HTTP/2 wants ONE. Domain sharding was clever under 1.1 and is actively harmful here.

▸ Why not just gzip the headers?

Because compressed size leaks the secret.

```
// the attacker cannot decrypt anything. it only watches SIZE.

GET /aaaa?guess=a   Cookie: session=SECRET…
   -> compressed length 412        no repeat found
GET /aaaa?guess=b   Cookie: session=SECRET…
   -> compressed length 412        no repeat found
GET /aaaa?guess=S   Cookie: session=SECRET…
   -> compressed length 411        <- the compressor found "S" twice

// one byte smaller means "correct". repeat per character.
```

HPACK's defence is structural

The Huffman table is FIXED — built once from a corpus, published in Appendix B, never adapted to the data flowing through it. There is no shared adaptive dictionary for attacker text and secret text to meet in.

The dynamic table is an exact-match index, not a substring matcher. Guessing a prefix of your cookie wins nothing.

RFC 7541 §7.1 goes further: mark sensitive fields "never indexed".

Yours too: gzip a JSON response containing a user-supplied search term and an API key together, and you have rebuilt CRIME.

▶ Same six requests. One connection. 400× faster.

HTTP/1.1 pipelined

slide 23, one connection

```
/big.txt?delay=2.0   2003.4 ms
/style.css           2003.6 ms
/app.js              2003.8 ms
/logo.png            2004.1 ms
/icon.png            2004.2 ms
/hero.png            2004.4 ms
```

Responses must arrive in request order, because order is the only identifier there is.

HTTP/2

one connection — the SAME one

```
stream  3  /style.css      4.9 ms
stream  7  /logo.png       5.7 ms
stream  9  /icon.png       6.9 ms
stream  5  /app.js         6.9 ms
stream 11  /hero.png       7.8 ms
stream  1  /big.txt     2005.9 ms
```

Odd ids, client-initiated. Arriving out of order, on purpose, because each frame says who it is for.

and the dynamic table warming up in real time

```
-> HEADERS stream=1  GET /big.txt?delay=2.0   84B of text -> 30B hpack
-> HEADERS stream=3  GET /style.css          76B of text -> 12B hpack
-> HEADERS stream=5  GET /app.js             73B of text -> 11B hpack
-> HEADERS stream=7  GET /logo.png           75B of text -> 12B hpack
```

Same GET. Same 200. Same ETag. Our h2 handler is line-for-line the shape of the HTTP/1.1 one — that is RFC 9110 doing its job.

A protocol can only give you permission

Handling requests inline in the read loop

what I wrote first

```
for event in conn.receive_data(data):
    if isinstance(event, RequestReceived):
        self.serve(event.stream_id, ...)
```

```
stream 1   /big.txt?delay=2  2005.6 ms
stream 3   /style.css        2005.7 ms
stream 5   /app.js           2005.7 ms
stream 7   /logo.png         2005.8 ms
stream 9   /icon.png         2005.8 ms
stream 11  /hero.png         2005.9 ms
```

Exactly the HTTP/1.1 numbers.

One thread per stream

and a lock around the connection

```
threading.Thread(target=self.safe_serve,
                 args=(event.stream_id, ...),
                 daemon=True).start()
```

```
stream 3   /style.css            4.9 ms
stream 7   /logo.png             5.7 ms
stream 9   /icon.png             6.9 ms
stream 5   /app.js               6.9 ms
stream 11  /hero.png             7.8 ms
stream 1   /big.txt?delay=2   2005.9 ms
```

Same frames. Same client. Same wire.

You will meet this exact shape again: a worker pool sharing one database connection · a blocking call inside a coroutine · a gRPC handler holding a global mutex · six goroutines waiting on one channel. The transport is fine. You serialised the work yourself.

▸ Server push: nobody deprecated it, everybody removed it

What the RFC says

`still standard, today`

Retained in RFC 9113 §8.4.
Retained in RFC 9114 §4.6 (HTTP/3).

Search RFC 9113 for "deprecat" and you get exactly two hits, neither of them push:

- `RFC 7540 PRIORITY` (§2, §5.3)
- `the h2c Upgrade token` (§3.1)

If you have heard "the RFC deprecated server push" — it did not.

What implementers did

`Chrome 106, 27 September 2022`

1.25%
of HTTP/2 sites used it. Later 0.7%.

"no clear net performance gain and in many cases performance regressions"

The fundamental problem: the server does not know what is in your cache. So it pushes a file you already have — spending bandwidth AND competing with what you actually needed, on the same connection and congestion window.

The replacement: 103 Early Hints (RFC 8297). Do not push the file — say "you are going to want style.css". The client checks its own cache and decides. **The server keeps the knowledge; the client keeps the decision.** When one side has the knowledge and the other has the state, inform — do not act.

The one thing HTTP/2 could not fix

> "because the parallel nature of HTTP/2's multiplexing is not visible to TCP's loss recovery mechanisms, a lost or reordered packet causes all active transactions to experience a stall regardless of whether that transaction was directly impacted by the lost packet."
>
> RFC 9114 §1.1

```
$ h2_client.py --port 9020 --multiplex   # through tools/stall_tcp.py
  stall: holding the stream at byte 3000 for 300 ms

  run 1   logo 4.5  style 4.7  | app 306.3  hero 307.0  icon 307.3  big 308.9
  run 2   style 4.0  icon 5.2  | logo 308.2  big 308.3  hero 308.4  app 308.4
  run 3   style 3.5            | big 306.4  app 306.5  icon 306.5  logo 306.7
                                                          hero 306.8
          ^ through before      ^ everything behind the hole, whether or not
            the hole              it lost a single byte
```

The later bytes had ARRIVED — they were in the receive buffer. The kernel may not hand them over without breaking the in-order guarantee that is the reason you chose TCP. And WHICH streams get caught changes every run: the blast radius is "whoever had bytes behind the gap", and you do not get to choose.

Google's SPDY data: 11.81% speedup at 0% loss, 47.7% at 2% loss. Multiplexing helps most exactly where TCP hurts most.

▶ UDP, because the internet stopped accepting new protocols

The obvious answer, and why it fails

```
TCP = 6    UDP = 17    SCTP = 132
```

Just take protocol number 253 and build a proper transport.

Except: every NAT, firewall, CGNAT, corporate proxy and hotel router on earth drops what it does not recognise.

SCTP is a good protocol, standardised in 2000, and you cannot use it across the public internet. That is why.

UDP is not a good foundation.
It is a hole that already exists.

And so: encrypt almost everything

"middleboxes could observe traits of a specific version of QUIC and assume that when other versions exhibit similar traits the same underlying semantic is being expressed." — RFC 8999 §7

Anything a middlebox CAN read, it will eventually depend on — and then you can never change it.

That is how TCP options became unusable. QUIC encrypts packet numbers and most of the header — not only for privacy, but to keep the protocol changeable.

And it lives in userspace. A TCP change is a kernel upgrade on a billion machines and a decade. A QUIC change ships with your app.

One handshake, not two

TCP + TLS 1.3 + HTTP/2

three round trips, cold

```
SYN  ->
     <-  SYN/ACK
ACK  ->                        1 RTT
     ...nothing has been said yet.

ClientHello ->    (ALPN rides here)
            <-    ServerHello …
Finished    ->                 2 RTT

GET  ->
     <-  200                   3 RTT
```

With TLS 1.2 it was four.

QUIC + HTTP/3

two round trips, cold

```
Initial  ->   TLS ClientHello inside
              a QUIC CRYPTO frame
         <-   Handshake              1 RTT

GET      ->
         <-   200                    2 RTT
```

RFC 9001 §2.1: there is no transport handshake and then a crypto handshake. There is one handshake.

And TLS is not optional. There is no unencrypted QUIC.

```
$ python3 05-http3/handshake_race.py --rtt 150

                                          measured   round trips
  TCP + TLS 1.3 + h2 (new connection)      456.6 ms          3.0
  QUIC + HTTP/3 (new connection, 1-RTT)    319.8 ms          2.1
```

Layering is how we make systems comprehensible — and every layer boundary is a place you may pay a round trip.

▶ Zero round trips, and the price of them

```
$ python3 05-http3/handshake_race.py --rtt 150

                                          measured   round trips
  TCP + TLS 1.3 + h2 (new connection)      456.6 ms          3.0
  QUIC + HTTP/3 (new connection, 1-RTT)    319.8 ms          2.1
  QUIC + HTTP/3 (0-RTT, request in flight 1)  161.7 ms       1.1

  Cold connection: QUIC saves 137 ms.  Resumed: 0-RTT saves 295 ms.
```

RFC 9001 §9.2, as written

"use of TLS early data comes with an exposure to replay attack… Endpoints MUST implement and use the replay protections described in [TLS13], however it is recognized that these protections are imperfect."

Imperfect because the defence is a single-use ticket cache, and a server farm has many servers. An attacker who captured your early data sends it to a different one.

So the rule is yours to make

"An application protocol that uses QUIC MUST describe how the protocol uses 0-RTT and the measures that are employed to protect against replay attack."

In practice: idempotent requests only. GET is fine; a POST that charges a card is not. Cloudflare allows only GET and HEAD, and sets an Early-Data header so your origin can refuse with `425 Too Early`.

0-RTT is a cache of a negotiation. Every cache of a negotiation trades a round trip for the possibility of a replay.

▶ Only the stream that lost data waited

HTTP/2 over TCP

one 300 ms stall in the byte stream

```
/logo.png       4.5 ms   before the hole
/style.css      4.7 ms   before the hole
/app.js       306.3 ms
/hero.png     307.0 ms
/icon.png     307.3 ms
/big.txt      308.9 ms
```

Streams that lost nothing, stalled — and a different set of them each run.

TCP cannot see your streams. One ordered byte pipe, one gap, everything behind it waits.

HTTP/3 over QUIC

18 datagrams really deleted (5%)

```
/style.css     4.5 ms
/app.js        5.4 ms
/logo.png      6.7 ms
/icon.png      7.8 ms
/hero.png     11.0 ms
/big.txt      78.2 ms   <- lost packets
```

Only the stream that lost data waited.

Every STREAM frame carries a stream id and an offset. Ordered within a stream, unordered between them.

Be precise about this experiment: the TCP side delays a chunk (the shape a lost segment makes); the QUIC side really deletes 18 datagrams. Different perturbations — do not read 307 vs 78 as a benchmark. What is comparable is the blast radius: on TCP it is "whoever had bytes behind the gap", and it is a different set every run. On QUIC it is exactly one stream, every time.

▸ Your connection survives changing networks

TCP: the connection IS the four-tuple

`(src IP, src port, dst IP, dst port)`

Change any one of them and it is a different connection. Which is to say: it is a dead connection.

Walk out of the coffee shop:

```
wifi -> LTE, your IP changes
every TCP connection dies
video buffers, download restarts,
websocket reconnects, spinner
```

Same thing when your NAT quietly rebinds your external port after idle.

QUIC: identity is not addressing

"The primary function of a connection ID is to ensure that changes in addressing at lower protocol layers (UDP, IP) do not cause packets for a QUIC connection to be delivered to the wrong endpoint." §5.1

New address, same connection id, path validated, connection carries on.

And the abuse case, handled in the same section: a stable visible id is a tracking identifier. So ids "MUST NOT contain any information that can be used by an external observer to correlate them" — endpoints issue a pool and rotate.

Identity belongs at the layer that owns the semantics. TCP made addressing and identity the same thing, and inherited every problem mobility causes.

HPACK had its own head-of-line blocking

> "If HPACK were used for HTTP/3, it would induce head-of-line blocking for field sections due to built-in assumptions of a total ordering across frames on all streams." — RFC 9204

The problem QUIC created

HPACK's dynamic table is shared mutable state. "Field number 67" only means something if you have already processed every insertion that made 67 mean it.

On TCP that is free — total ordering.

QUIC deliberately removed exactly that guarantee. A block on stream 12 could reference an entry inserted on stream 8 that has not arrived. So: stall stream 12.

QPACK's fix

take the mutable state off the request streams

```
encoder stream  (type 0x02)
   encoder -> decoder: table insertions
decoder stream  (type 0x03)
   decoder -> encoder: "I have seen these"
```

The encoder only references entries it knows are safe.

```
QPACK_BLOCKED_STREAMS     default 0
QPACK_MAX_TABLE_CAPACITY  default 0
```

Static table: HPACK has 61 entries from 1; QPACK has 99 from 0, rebuilt on newer traffic — the sec-fetch headers are in it.

Shared mutable state and out-of-order delivery do not mix. Relax an ordering guarantee, then go and find what was relying on it.

How do you find it, and what if UDP is blocked?

Mechanism	RFC · date	How it works	What it costs
ALPN	RFC 7301 · Jul 2014	Protocol chosen inside the TLS handshake — "h2", "h3"	Nothing. But it needs a handshake first.
Alt-Svc	RFC 7838 · Apr 2016	A response header: h3=":443". The browser remembers.	One full TCP+TLS connection, every cold cache
HTTPS DNS RR	RFC 9460 · Nov 2023	Record type 65 carries the ALPN list in the DNS answer	Nothing — you were doing the lookup anyway

3–5%

of networks block all UDP traffic
(RFC 9308 §2, quoting measurement studies)

95.3%

of Google's clients could use QUIC;
4.4% blocked or packet-size problems

downgrade

drop the UDP and wait for the client to
fall back to TCP by itself

RFC 9308 §2: "A fallback to TCP and TLS exposes control information to modification and manipulation in the network." Any protocol with a fallback path has, by construction, a downgrade attack. The only question is what the fallback costs you.

Where the web actually is

Source	Measurement	Figure	Read it how?
HTTP Archive Web Almanac 2024	By request, real crawl	HTTP/1.1 ≈15% · HTTP/2+ ≈85%	Cleanest number here
HTTP Archive Web Almanac 2024	By homepage	h1 21–22% · h2 70–71% · h3 7–9%	Cleanest number here
HTTP Archive Web Almanac 2024	Advertising h3	Alt-Svc ≈26–28% · DNS HTTPS RR ≈9–10%	The DNS record is the laggard
W3Techs 3 Sep 2026	By website	HTTP/2 34.6% · HTTP/3 40.3%	Treat with suspicion — h3 > h2 means the categories are not what they look like

Fifteen per cent of requests are still HTTP/1.1, in 2026, eleven years after HTTP/2 shipped. Internal services, embedded devices, that one load balancer nobody wants to touch, a thousand APIs behind a gateway. You will write HTTP/1.1 parsers in your career. There is no cutover — there is a very long tail, and most engineering happens at the boundary between versions rather than inside one.

▶ Homework

#	Do this	Until you can explain
1	Extend fetch_page.py to N connections. Plot wall time against N at 150 ms RTT for 6, 20 and 60 assets. Find the knee.	Why the knee is where it is — and it is not "you ran out of connections". Read cwnd in ss -ti.
2	Make server11.py build its ETag from mtime, then modify a file twice within one second.	Which of RFC 9110 §8.8.1's two validators you just built, and why Range requests need the other one.
3	Decode a HEADERS frame's HPACK block by hand — three field lines, against RFC 7541 §6 — before letting hpack_mini.py do it.	Why the second identical request costs 6 bytes. You will never forget HPACK after this.
4	Set SETTINGS_QPACK_BLOCKED_STREAMS to 0, then to 16, in h3_server.py.	What the encoder may do differently in each case. The hardest one here.
5	iptables -A OUTPUT -p udp --dport 4433 -j DROP, then time the fallback. If you have tc netem, redo slide 41 with real loss on both sides.	What your own h3 timeout should be — and whether my caveat on slide 41 was too cautious, or not enough.

All five are in the repo README. Number three is the one to do if you only do one.

Four ideas

1 Multiplexing needs identity

HTTP/1.1 had no request id, so order became the identifier, and ordering became mandatory. 31 bits fixed it. You have now seen the same move as TCP ports, SS7 transaction ids, FastCGI request ids and QUIC stream ids. When a client must wait before sending the next message — check whether you left the identifier out.

2 Every fix is the next problem

Persistence made pipelining possible; pipelining created head-of-line blocking. HTTP/2 fixed it above TCP and exposed it inside TCP. QUIC fixed that and broke HPACK, which needed QPACK. There is no final version — only a sequence of trades, and knowing which one you are standing on.

3 Semantics and encoding differ

RFC 9110 is what HTTP means; 9112, 9113 and 9114 are three ways of writing it down. GET has not changed since 1996; the wire format has changed three times. Every long-lived system needs this seam — without it, a new encoding means rewriting your semantics.

4 Deployed reality beats specification

302 lost to browsers, so we got 307. Two connections lost to six, and the sentence was deleted. Postel's law lost to request smuggling. 418 was a joke that got reserved forever. Push was never deprecated and is dead. QUIC runs on UDP because middleboxes have a veto.

CN & SCALER · SESSION 6

Where did the state go?

Sessions and sticky routing · what "stateless" actually costs · TLS termination and where you put it · shared caches and the thundering herd · connection pools, which is amortise-the-setup for the sixth time

Bring this question

Today's connection can migrate across networks and survive a NAT rebind. Your session cookie cannot.

Which of those two is the real identity of a user?

`http-evolution.tar.gz`

5 servers · 5 clients · 4 proxies · ~2,900 lines · no network required.

`./setup.sh && ./run_all.sh`

reproduces every number on every slide today, in about 40 seconds.

Not covered, and worth your own time: TLS beyond ALPN · HTTP/2 priorities and RFC 9218 · WebTransport and QUIC datagrams · MASQUE · RFC 9111, the whole of caching

AGENDA

Four wire formats

Hour	Version	The RFC	Ends on
1	HTTP/1.0	RFC 1945 · May 1996	One response per connection
2	Early HTTP/1.1	RFC 2068 · Jan 1997	"At most 2 connections"
3	HTTP/1.1 now	RFC 2616 → 7230 → 9110	Head-of-line blocking
4	HTTP/2	RFC 7540 → 9113	TCP head-of-line blocking
5	QUIC + HTTP/3	RFC 9000 · RFC 9114	The 3–5% who block UDP

The protocol did not drift — it was pushed.